

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 4_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items whose total weight is exactly equal to a required target value. Each item can be selected at most once.

Given a list of item weights and a target weight K , determine whether it is possible to select a subset of items such that their sum is exactly K .

If such a subset exists, print any one valid subset. If no such subset exists, print -1.

This problem is derived from the Subset Sum problem, which is NP-Complete, and the solution must be implemented using Dynamic Programming.

Input Format

The first line contains two integers n and K

The second line contains n space-separated integers

Output Format

Print elements of one valid subset whose sum equals K

If no such subset exists, print -1

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 9
3 3 4 1 2 5
Output: 5 4

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    int n, K;
    scanf("%d %d", &n, &K);
    int arr[n];
    for(int i=0; i<n; i++) scanf("%d", &arr[i]);

    int dp[n+1][K+1];
    for(int i=0; i<=n; i++) {
        for(int j=0; j<=K; j++) {
            if(j==0) dp[i][j]=1;
            else if(i==0) dp[i][j]=0;
            else {
                dp[i][j]=dp[i-1][j];
                if(j>=arr[i-1]) dp[i][j] |= dp[i-1][j-arr[i-1]];
            }
        }
    }

    if(!dp[n][K]) {
```

```

    printf("-1\n");
    return 0;
}

int subset[n], idx=0;
int i=n, j=K;
while(i>0 && j>0) {
    if(dp[i-1][j]) {
        i--;
    } else {
        subset[idx++]=arr[i-1];
        j-=arr[i-1];
        i--;
    }
}

for(int k=0;k<idx;k++) printf("%d ",subset[k]);
printf("\n");
return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Rohit is a travel planner who must organize a business trip covering multiple cities.

The travel cost between each pair of cities is given in a matrix.

He must:

Start from city 0
Visit every city exactly once
Return back to city 0
Your task is to compute:

The minimum possible travel cost
The sequence of cities visited
The average cost per step (excluding the final return step)

This problem is a classic Travelling Salesman Problem (TSP) solved using Dynamic Programming with Bitmasking (Held–Karp Algorithm).

Input Format

- The first line contains an integer n , the number of cities.
- The next n lines contain n space-separated integers, representing the cost matrix.

$\text{cost}[i][j]$ is the cost of traveling from city i to city j .

Output Format

Print the path:

The Path is: $0 \rightarrow \dots \rightarrow 0$

Print the minimum cost:

Minimum cost is X

Print the average cost per step:

Average cost per step is Y

Rounded to 2 decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: The Path is: 0--->2--->3--->1--->0

Minimum cost is 80

Average cost per step is 26.67

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int cost[15][15];
```

```
int dp[1<<10][10];
```

```
int parent[1<<10][10];
```

```
int main(){
```

```
    scanf("%d",&n);
```

```
    for(int i=0;i<n;i++)
```

```
        for(int j=0;j<n;j++)
```

```
            scanf("%d",&cost[i][j]);
```

```
    int fullMask=(1<<n);
```

```
    for(int mask=0;mask<fullMask;mask++)
```

```
        for(int i=0;i<n;i++)
```

```
            dp[mask][i]=INT_MAX/2;
```

```
    dp[1][0]=0;
```

```
    for(int mask=1;mask<fullMask;mask++){
```

```
        for(int u=0;u<n;u++){
```

```
            if(!(mask&(1<<u))) continue;
```

```
            for(int v=0;v<n;v++){
```

```
                if(mask&(1<<v)) continue;
```

```
                int newMask=mask|(1<<v);
```

```
                if(dp[newMask][v]>dp[mask][u]+cost[u][v]){
```

```
                    dp[newMask][v]=dp[mask][u]+cost[u][v];
```

```
                    parent[newMask][v]=u;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int ans=INT_MAX/2,last=-1;
```

```

for(int i=1;i<n;i++){
    if(ans>dp[fullMask-1][i]+cost[i][0]){
        ans=dp[fullMask-1][i]+cost[i][0];
        last=i;
    }
}

int path[n+1],idx=n;
path[idx--]=0;
int mask=fullMask-1;
int cur=last;
while(cur!=0){
    path[idx--]=cur;
    int p=parent[mask][cur];
    mask^=(1<<cur);
    cur=p;
}
path[idx]=0;

printf("The Path is: ");
for(int i=0;i<=n;i++){
    printf("%d",path[i]);
    if(i<n) printf("--->");
}
printf("\n");

printf("Minimum cost is %d\n",ans);

double avg=(double)(ans-cost[last][0])/(n-1);
printf("Average cost per step is %.2f\n",avg);

return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic

knapsack with a maximum weight capacity of W . Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i -th integer represents the weight of the i -th item.

The third line contains n integers, where the i -th integer represents the value of the i -th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
```

```
int level;  
int value;  
int weight;  
double bound;  
} Node;
```

```
int n,W;  
int weights[105],values[105];
```

```
int cmp(const void *a,const void *b){  
    double r1=(double)values[* (int*)a]/weights[* (int*)a];  
    double r2=(double)values[* (int*)b]/weights[* (int*)b];  
    return r2>r1?-1:1;  
}
```

```
double bound(Node u,int order[]){  
    if(u.weight>=W) return 0;  
    double result=u.value;  
    int j=u.level+1;  
    int totweight=u.weight;  
    while(j<n && totweight+weights[order[j]]<=W){  
        totweight+=weights[order[j]];  
        result+=values[order[j]];  
        j++;  
    }  
    if(j<n) result+=(W-totweight)*(double)values[order[j]]/weights[order[j]];  
    return result;  
}
```

```
int knapsack(){  
    int order[105];  
    for(int i=0;i<n;i++) order[i]=i;  
    qsort(order,n,sizeof(int),cmp);
```

```
    Node u,v;  
    u.level=-1;u.value=0;u.weight=0;  
    int maxValue=0;  
    u.bound=bound(u,order);
```

```
    Node queue[10000];  
    int front=0,rear=0;  
    queue[rear++]=u;
```

```

while(front<rear){
    u=queue[front++];
    if(u.bound<=maxValue) continue;
    v.level=u.level+1;
    v.weight=u.weight+weights[order[v.level]];
    v.value=u.value+values[order[v.level]];
    if(v.weight<=W && v.value>maxValue) maxValue=v.value;
    v.bound=bound(v,order);
    if(v.bound>maxValue) queue[rear++]=v;
    v.weight=u.weight;
    v.value=u.value;
    v.bound=bound(v,order);
    if(v.bound>maxValue) queue[rear++]=v;
}
return maxValue;
}

```

```

int main(){
    scanf("%d %d",&n,&W);
    for(int i=0;i<n;i++) scanf("%d",&weights[i]);
    for(int i=0;i<n;i++) scanf("%d",&values[i]);
    int result=knapsack();
    printf("%d\n",result);
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Josh is learning about the Fractional Knapsack problem, where a knapsack has a fixed maximum capacity. Given a set of items, each with a value and a weight, Josh wants to maximize the total value placed into the knapsack.

Unlike the 0/1 Knapsack problem, fractions of items are allowed. Items must be selected using a Greedy strategy based on their value-to-weight ratio.

Write a program to determine the fraction of each item that should be

included in the knapsack to achieve the maximum total value.

Formula

Value-to-Weight Ratio

ratio = value / weight

Fraction Calculation

fraction = remainingCapacity / weight

Notes

Items must be selected using the Greedy approach based on value-to-weight ratio. Sorting is required to determine the optimal order. Output fractions must be printed in the original item order. Use double datatype only for calculations.

Input Format

The first line contains an integer n, representing the number of items.

The second line contains n space-separated integers representing item values.

The third line contains n space-separated integers representing item weights.

The fourth line contains an integer capacity, representing knapsack capacity.

Output Format

Print n space-separated double values, representing the fraction of each item selected.

Output must be printed rounded to exactly two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

3 5 1 2 4

40 50 20 10 30

75

Output: 0.00 0.70 0.00 1.00 1.00

Answer

```
// You are using GCC#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct {
    int index;
    int value;
    int weight;
    double ratio;
} Item;
```

```
int cmp(const void *a,const void *b){
    double r1=((Item*)a)->ratio;
    double r2=((Item*)b)->ratio;
    if(r1<r2) return 1;
    else if(r1>r2) return -1;
    else return 0;
}
```

```
int main(){
    int n,capacity;
    scanf("%d",&n);
    int values[n],weights[n];
    for(int i=0;i<n;i++) scanf("%d",&values[i]);
    for(int i=0;i<n;i++) scanf("%d",&weights[i]);
    scanf("%d",&capacity);
```

```
    Item items[n];
    for(int i=0;i<n;i++){
        items[i].index=i;
        items[i].value=values[i];
        items[i].weight=weights[i];
        items[i].ratio=(double)values[i]/weights[i];
    }
```

```
    qsort(items,n,sizeof(Item),cmp);
```

```
    double fractions[n];
    for(int i=0;i<n;i++) fractions[i]=0.0;
```

```

int remaining=capacity;
for(int i=0;i<n && remaining>0;i++){
    if(items[i].weight<=remaining){
        fractions[items[i].index]=1.0;
        remaining-=items[i].weight;
    } else {
        fractions[items[i].index]=(double)remaining/items[i].weight;
        remaining=0;
    }
}

for(int i=0;i<n;i++){
    printf("%.2f",fractions[i]);
    if(i<n-1) printf(" ");
}
printf("\n");
return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Ananya is preparing for a long trek and has a bag with limited capacity. She has several items, each with a weight and value, and she wants to maximize the total value she can carry. She may take whole items or fractions of them to fit in her bag.

The maximum value is calculated using the formula:

Maximum Value = $\sum (\text{value}[i] * \text{fraction_taken}[i])$

where $\text{fraction_taken}[i] = \min(1, \text{remaining_capacity} / \text{weight}[i])$

Write a program to help Ananya determine the maximum achievable value.

Input Format

The input consists of several lines, each representing an item.

Each line (until -1) contains two integers: weight and value of an item.

The input terminates with -1.

The final line contains one integer representing the bag's capacity.

Output Format

The output prints "The maximum value of the current list is:"

On the next line, print the maximum value achievable with exactly two decimal places

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 10 60

20 100

30 120

-1

50

Output: The maximum value of the current list is:

240.00

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    int weight;
```

```
    int value;
```

```
    double ratio;
```

```
} Item;
```

```
int cmp(const void *a,const void *b){
```

```
    double r1=((Item*)a)->ratio;
```

```
    double r2=((Item*)b)->ratio;
```

```
    if(r1<r2) return 1;
```

```
    else if(r1>r2) return -1;
```

```
    else return 0;
```

```

}

int main(){
    Item items[1005];
    int n=0;
    while(1){
        int w;
        if(scanf("%d",&w)!=1) return 0;
        if(w==-1) break;
        int v;
        scanf("%d",&v);
        items[n].weight=w;
        items[n].value=v;
        items[n].ratio=(double)v/w;
        n++;
    }
    int capacity;
    scanf("%d",&capacity);

    qsort(items,n,sizeof(Item),cmp);

    double maxValue=0.0;
    int remaining=capacity;
    for(int i=0;i<n && remaining>0;i++){
        if(items[i].weight<=remaining){
            maxValue+=items[i].value;
            remaining-=items[i].weight;
        } else {
            maxValue+=items[i].value*((double)remaining/items[i].weight);
            remaining=0;
        }
    }

    printf("The maximum value of the current list is:\n");
    printf("%.2f\n",maxValue);
    return 0;
}

```

Status : Correct

Marks : 10/10